7.6 Regulating Databases: Key Constraints

Key Constraints: Why and What?

- Some attributes serve as keys - or identifiers e.g. SSN, email.
- DB needs to know such attributes, to ensure their correctness, and to enhance performance. (e.g. creating indexes)
- Types of keys:
  - Primary Key: Attributes (e.g. SSN) as a primary way to identify a tuple.
    - SQL Keywords: PRIMARY KEY
  - Unique Key: Attributes (e.g. email) as a way to uniquely identify a tuple.
    - SQL Keywords: UNIQUE
  - Foreign Key: Attributes that uniquely identify a tuple in another table
    - SQL Keywords: FOREIGN KEY, REFERENCES

PRIMARY KEY vs UNIQUE KEY Constraints

- Both have in common
  - Values are unique
- Differences
  - PRIMARY KEY: at most one.
    UNIQUE: can be more than one.
  - PRIMARY KEY: values are also NOT NULL
    UNIQUE: values can be NULL.
  - SQL standard allows DBMS to make own distinctions.
    - E.g. MySQL automatically creates an index for primary key.

FOREIGN Keys

- Foreign Key is a set of one or more attributes of a table that refers to a key in another table.
- Foreign-key (referential integrity) constraint requires a set of attributes to refer to key values in another table.
- Consider Relation Sells(bar, beer, price).
  - We expect that a beer value is a real beer - so it should appear in Beers.name. Thus, Sells.beer is a foreign key of Sells.
  - A constraint that requires a beer in Sells to be some key in Beers is called a foreign-key constraint.

Expressing Foreign Keys When Creating Tables

- In an attribute element:
  <att> <type> REFERENCES <table> (<attribute>)
- As a separate element:
  FOREIGN KEY (<attributes>)
  REFERENCES <table> (<attributes>)
- Referenced attributes must be declared PRIMARY KEY or UNIQUE in the referenced table.

How can a Foreign-Key Constraint Be Violated?

- Consider a foreign-key constraint S.a references T.k.
- Violation 1: Source Change
  - An insertion or update to S.a introduces values not found in T.
- Violation 2: Target Change.
  - A deletion or update to T.k causes some tuples of S to "dangle".

Actions for Handling Violations

- Violation 1: Source change - Must be rejected.
  - The change of source introduces a non-existent key for Target.
  - Such violations can only be removed by rejecting the change.
- Violation 2: Target change - Three choices.
  - The change of Target removed a key that Source referenced.
    1. Reject (default): Reject the modification.
    2. Cascade: Change S.a in the same way (delete or update) as T.k.
    3. SET NULL: Change S.a to NULL.

Declaring Actions for Foreign-Key Constraints

- Add declaration ON <DELETE/UPDATE> <ACTION>
  - E.g., ON DELETE CASCADE
  - E.g., ON UPDATE SET NULL
- If declaration is omitted, the default (reject) is taken.

7.7 Regulating Databases: Checks and Assertions

General Constraint Mechanisms

- Checks
  - Attribute-based checks
  - Tuple-based checks
- Assertions
  - Database-based
- Advanced features - not uniformly supported in various RDBMS.
  - MySQL does not support checks and assertions.
  - PostgreSQL supports checks without subqueries in checks.

Checks 1: Attribute-based

- Constrain the value of a particular attribute in a tuple.

```
CREATE TABLE <name>(
    ....,
    <attr> <type> CHECK (<condition>)
    ....);
```

- The condition refers to the attribute; any other attributes or relations must be in a subquery to compare.
  - E.g. price float CHECK (price > 1.0)
- Timing: Attribute-based check is checked only when the attribute is inserted or updated.

Checks 2 Tuple-based

- Constrain the value of multiple attributes in a tuple.
  - CREATE TABLE <name> (
    ....,
    CHECK (<condition>),
    ....);
- The condition refers to multiple attributes in the table; any other attributes or relations must be in a subquery to compare.
  - E.g. CHECK (price * (1 - discount) > 1.0)
- Timing: Tuple-based check is checked only when a tuple is inserted or updated.

Assertions: Database-based

- We declare an assertion as part of the database schema:
  CREATE ASSERTION <name> CHECK (<condition>);
- Thus, a CHECK over the database
  - Not limited to a tuple.
  - Condition may refer to any relation or attribute in the database schema.

```
CREATE ASSERTION NoBoringBars CHECK (
    NOT EXISTS(
        SELECT bar FROM Sells
        GROUP BY bar
        HAVING COUNT(beer) < 3
));
```

7.8 Regulating Databases: Triggers

Checks and Assertions Fall Short

- Limitations:
  - Lacking timing
    - Require checking whenever any updates are made (on the elements involved) - expensive!
  - Lacking flexibility
    - Checks are limited to attributes or tuple-based constraints.
  - Lacking handling
    - We cannot control what to do when constraints are violated.
- What's missing?
  - Timing
  - Flexibility
  - Handling

Trigger: Event-Condition-Action Rules

- Trigger is also called ECA rule, event-condition-action rule.
- Event: provide timing
  - change of database state, i.e. modification
    e.g. "insert on Sells"
- Condition: provide flexibility
  - Any SQL Boolean-valued expression, with flexible scopes.
- Action: provide handling
  - Any SQL (modification) statements.
- SQL standard since SQL:1999.
  - Widely implemented.
  - However, with slightly non-uniform functionality and syntax.

Trigger: Example #1

```
CREATE TRIGGER PriceReset
  AFTER
     UPDATE OF price ON sells
  REFERENCING
     OLD ROW AS old
     NEW ROW AS new
  FOR EACH ROW
  WHEN (new.price > old.price + 5.00)
       UPDATE sells SET price = 0
          WHERE bar = new.bar and beer =
          new.beer
```

Trigger 1): Event

- Describes timing of trigger activation
- Consists of preposition + event
- Preposition:
  - AFTER, BEFORE
  - INSTEAD OF
- Event: A modification to the database
  - INSERT, DELETE of tuples
  - UPDATE on attributes

Trigger 2): Condition

- Describes a condition to check
  REFERENCING.....
  FOR EACH ROW / STATEMENT
  WHEN <condition>
- Variables: REFERENCING declares data elements to refer to.
- Scope:
  - row-level: FOR EACH ROW
  - statement-level: FOR EACH STATEMENT
- Constraint violation: as expressed in WHEN <condition>

Trigger 2): Condition - REFERENCING

- Declares variables to refer to.
  REFERENCING [NEW/OLD] [ROW/TABLE] AS <name>
- Row level
  - INSERT → NEW ROW
  - DELETE → OLD ROW
  - UPDATE → both OLD ROW and NEW ROW
- Statement level
  - Both NEW TABLE and OLD TABLE

Trigger: Example #2

```
CREATE TRIGGER BeerForeignKey
  AFTER
    INSERT ON sells
  REFERENCING
     NEW ROW AS new
  FOR EACH ROW
  WHEN (new.beer NOT IN (SELECT name FROM
        Beers))
    INSERT INTO Beers(name) VALUES(new.beer
```

Trigger: Example #3

```
CREATE TRIGGER BarShouldNotBeBoring
  AFTER
    DELETE OR UPDATE ON sells
  FOR EACH STATEMENT
    WHEN EXISTS (SELECT bar FROM sells
                 GROUP BY bar HAVING
                 COUNT(beer) < 3)
    INSERT INTO Boring(bar) VALUES
       (SELECT bar FROM sells GROUP BY bar
          HAVING COUNT(beer) < 3));
```